AI Job Application Assistant

Project documentation, phased roadmap and build prompts

A free, personal, semi-automatic tool that reads a resume, finds matching jobs, tailors answers and pre-fills applications for human approval.

Prepared for	Ashish Pathak
Date	25 October 2026
Budget	Zero. Free tools and free API tiers only
Phases	4
Estimated effort	3 to 5 weeks, part-time

1. Project overview

The goal is a personal tool that understands your resume, collects suitable job openings from free sources, ranks them, prepares a tailored application for each good match and fills in the application form in your browser. You review the filled form and press Submit yourself. Later, once the tool is proven, submission can be automated for a small set of safe job sites.

Why semi-automatic instead of fully automatic

- **Account safety.** LinkedIn, Indeed and Naukri prohibit bots. Detection usually leads to a restricted or banned account.
- **CAPTCHAs and logins.** Big job sites block scripted browsers, and a free tool cannot reliably get past that.
- **Quality.** Blind mass applications are ignored by recruiters. A tailored, reviewed application performs better.
- **Mistakes are permanent.** A wrong answer to salary or notice period cannot be taken back after submission.

What the finished system does

- Reads your resume (PDF) and builds a structured profile of skills, experience, education and target roles.
- Collects jobs every day from legal, free sources: Greenhouse and Lever public job boards, the Adzuna API, remote-job feeds and a list of company career pages you choose.
- Scores every job from 0 to 100 against your profile and filters out poor matches.
- Writes a short, honest cover note and answers to common application questions, using only facts from your resume.
- Opens the application page in a visible browser window, fills the form and waits for your approval.
- Records every job and its status (new, prefilled, applied, skipped, interview, rejected) in a local database and shows it on a dashboard.

Out of scope (by design)

- Scraping or automating LinkedIn, Indeed or Naukri pages.
- Solving CAPTCHAs or evading bot detection.
- Inventing experience, skills or achievements that are not in your resume.
- Anything that costs money.

2. Free tech stack and requirements

Part	Tool	Cost
Language	Python 3.11+	Free
Resume parsing	pdfplumber or PyMuPDF	Free
AI (pick one)	Google Gemini free tier, Groq free tier, or Ollama running a local model (Llama 3 or Qwen, needs about 8 GB RAM)	Free, rate limited
Matching without AI	sentence-transformers embeddings, or TF-IDF cosine similarity	Free, offline
Browser automation	Playwright (Chromium)	Free
Database	SQLite	Free
Dashboard	Streamlit	Free
Config and secrets	YAML config file and a .env file (never committed to git)	Free
Hosting	Your own laptop	Free

Before you start

- A laptop with Python 3.11+, Chrome and VS Code installed.
- Your resume as a PDF, plus a plain-text copy for testing.
- A free Gemini or Groq API key, or Ollama installed locally.
- A GitHub repository. Add .env, the database file and your resume to .gitignore.
- Basic Python and some HTML and CSS selector knowledge, which your internships already cover.

Free job sources

Source	How to use it	Notes
Greenhouse and Lever	Public job-board JSON endpoints for each company	Most reliable and most legitimate source
Adzuna	Free developer API, covers India	Needs a free API key
RemoteOK, Remotive, We Work Remotely	Free feeds or APIs	Remote roles only
Company career pages	A watchlist of URLs you choose	Needs a small parser per site
LinkedIn, Indeed, Naukri	Use their email job alerts and read the links from your inbox	Do not scrape or automate these sites

3. Architecture and how it works

The system is a pipeline. Each stage reads from and writes to the same SQLite database, so any stage can be re-run on its own.

Stage	Input	Output
1. Profile builder	Resume PDF and config.yaml	profile.json
2. Job collector	Source list in config	Rows in the jobs table (deduplicated)
3. Matcher	profile.json and jobs	Score, reasons and gaps for each job
4. Tailor	Top-scoring jobs and profile	Cover note and answers for each job
5. Application assistant	Approved job and drafts	Browser form filled and waiting for you
6. Dashboard and scheduler	Database	Review screen, daily runs, status tracking

Suggested folder structure

```
job-assistant/
  config.yaml            # preferences, limits, sources
  .env                   # API keys (not in git)
  data/
    resume.pdf
    profile.json
    jobs.db
  src/
    profile/             # resume parser and profile builder
    collectors/          # greenhouse.py, lever.py, adzuna.py, remoteok.py
    matching/            # scoring and filters
    tailor/              # cover notes and answer drafts
    apply/               # playwright form filler, one module per ATS
    llm/                 # provider wrapper (Gemini, Groq, Ollama)
    db/                  # SQLite models
  app.py                 # Streamlit dashboard
  scheduler.py           # daily run
  README.md
```

Core safety rules (apply to every phase)

- A human approval step is mandatory before any submission in Phases 1 to 3.
- The AI may only use facts from profile.json and config.yaml. If a fact is missing, it must leave the field blank and flag it.
- A daily cap on applications (default 10) is enforced in code, not just in the prompt.
- Every action is logged with a timestamp so you can audit what the tool did.
- The LLM layer is behind one wrapper so you can switch providers if a free tier changes.

4. Master prompt (paste this first)

Paste this once at the start of your coding session (in VS Code with an AI assistant, or in a chat). It sets the rules for the whole project. Then paste one phase prompt at a time.

```
You are a senior Python engineer helping me build a personal, free, semi-automatic
job application assistant. I am a fresh computer science graduate looking for jobs.
Build it in phases. Do only the phase I ask for, and stop when it is finished.

GOAL
A local tool that (1) reads my resume PDF and builds a structured profile,
(2) collects job openings from free legal sources, (3) scores each job against my
profile, (4) drafts a tailored cover note and answers to common application
questions, (5) opens the application page in a visible browser, fills the form and
WAITS for my approval before submitting, and (6) tracks everything in a database
with a dashboard.

HARD CONSTRAINTS
- Everything must be free. No paid APIs, no subscriptions. Use free tiers only.
- Stack: Python 3.11+, SQLite, Playwright, Streamlit, pdfplumber or PyMuPDF.
- AI provider is one of Gemini free tier, Groq free tier, or local Ollama. Put it
  behind a single wrapper class so the provider can be swapped by changing config.
- Do NOT scrape or automate LinkedIn, Indeed or Naukri. Do NOT solve CAPTCHAs or
  try to evade bot detection.
- The AI must never invent experience, skills, degrees, dates or achievements.
  It may only use facts in profile.json and config.yaml. If information is missing,
  leave the field empty and flag it for me.
- Never submit an application without my explicit approval in the UI.
- Enforce a daily application limit from config.yaml in code.
- Store secrets in .env and keep them out of git. Add a .gitignore.
- Handle API rate limits with retries and backoff, and log all actions.

CODE QUALITY
- Clear folder structure, type hints, docstrings, small functions.
- Config-driven behaviour (config.yaml), no hardcoded personal data.
- Include a README with setup steps, how to run each phase, and troubleshooting.
- Add basic tests for parsing, scoring and deduplication.
- After finishing a phase, give me: what was built, how to run it, how to test it,
  and what is left for the next phase.

Acknowledge these rules and wait for me to say which phase to build.
```

Phase 1: Foundation: profile, job collector, database

Goal	Turn your resume into structured data and fill a local database with fresh, deduplicated jobs.
Estimated time	Week 1 (about 5 to 7 days)

What gets built

- Project skeleton, config file and secrets handling.
- LLM wrapper that works with Gemini, Groq or Ollama.
- Resume parser that produces and validates profile.json.
- Collectors for Greenhouse, Lever, Adzuna and RemoteOK.
- SQLite database with deduplication and a simple command line.

Done when

- profile.json accurately reflects your resume and you have reviewed it.
- Running collect fills the database with real jobs and re-running it adds no duplicates.
- Tests pass.

Phase 1 prompt

Paste this after the master prompt.

```
Build PHASE 1: Foundation (resume profile, job collector, database).

1. Project setup
- Create the folder structure: config.yaml, .env.example, .gitignore, data/, src/
  with profile/, collectors/, llm/, db/, plus README.md and requirements.txt.
- config.yaml must contain: target_roles, preferred_locations, experience_level,
  excluded_keywords, daily_application_limit, expected_salary, notice_period,
  work_authorization, phone, email, linkedin_url, github_url, portfolio_url,
  and a list of job sources with enable/disable flags and company slugs.

2. LLM wrapper (src/llm/)
- One class with a method generate(prompt, json_mode=False) and implementations
  for Gemini, Groq and Ollama, chosen by config. Include retry with exponential
  backoff and a clear error when the key is missing.

3. Resume profile builder (src/profile/)
- Extract text from data/resume.pdf using pdfplumber (fallback PyMuPDF).
- Use the LLM to convert it to structured JSON with: name, contact, summary,
  skills (grouped), education, experience (company, role, dates, bullets),
  projects, certifications, and inferred target roles.
- Validate the JSON against a schema, save to data/profile.json, and print a
  readable summary so I can check it. Make it easy for me to hand-edit the file
  and re-use my edits (do not overwrite manual edits without asking).

4. Job collectors (src/collectors/)
- Implement collectors for: Greenhouse public boards, Lever public boards,
  Adzuna API, and RemoteOK. Each returns a list of Job objects with the same
  fields: id, source, title, company, location, remote flag, description,
  apply_url, posted_date, fetched_at.
- Use only public, documented endpoints. Respect rate limits and add timeouts.

5. Database (src/db/)
- SQLite tables: jobs, matches (job_id, score, reasons, gaps), applications
  (job_id, status, cover_note, answers_json, updated_at), run_log.
- Deduplicate jobs by (source, id) and by a hash of company + title + location.

6. Command line
- python -m src.cli build-profile
- python -m src.cli collect
- python -m src.cli list-jobs --limit 20

Add tests for the resume parser output schema and for deduplication.
Stop after this phase and summarise how to run and test it.
```

Phase 2: Matching and tailoring

Goal	Rank jobs by fit and prepare honest, role-specific application drafts for the good ones.
Estimated time	Week 2 (about 5 to 7 days)

What gets built

- Hard filters, semantic scoring and a limited LLM review of the top jobs.
- Cover note and question-answer drafts per job, restricted to facts in your profile.
- A fact-check pass that rejects drafts containing invented details.
- Support for more than one resume version (for example technical and management).

Done when

- Running match gives a sensible ranked list and you agree with most of the top 10.
- Drafts read naturally, contain no invented facts, and unknown items show as [NEEDS INPUT].
- Free-tier API usage stays within limits on a normal day.

Phase 2 prompt

Paste this after the master prompt.

```
Build PHASE 2: Matching and tailoring. Phase 1 is complete and working.

1. Matching engine (src/matching/)
- Step A, hard filters from config.yaml: location or remote preference, experience
  level (reject jobs asking for far more experience than I have), excluded
  keywords, and companies on a blocklist.
- Step B, semantic score: compute embeddings (sentence-transformers, small model
  such as all-MiniLM-L6-v2, running locally) for my profile and each job
  description, then cosine similarity. Combine with a keyword overlap score on
  skills. Output a final score from 0 to 100 with a weighted formula that is
  defined in config.yaml.
- Step C, LLM review only for the top N jobs (N in config, default 20) to stay
  inside free-tier limits. For each, produce: a two-line "why it fits", a list of
  missing skills, and a verdict of strong / maybe / skip.
- Save results in the matches table. Cache everything so nothing is recomputed
  unless the profile or job changed.

2. Tailoring (src/tailor/)
- For each job with verdict strong or maybe, generate:
  a) a cover note of 120 to 160 words, specific to the company and role,
  b) answers to common questions: why this role, why this company, strengths,
     relevant project, availability, expected salary (from config), notice period
     (from config), work authorization (from config).
- Strict rules in the prompt: use only facts from profile.json; no invented
  numbers or tools; if something is not in my data, output the placeholder
  [NEEDS INPUT] instead of guessing.
- Run a second check that scans the draft and rejects it if it contains skills,
  employers or numbers that are not in my profile.
- Save drafts in the applications table with status "drafted".

3. Resume variants
- Support multiple resume files in config (for example full_stack.pdf and
  management.pdf). Choose the best one per job based on the role title and the
  match, and record which one was chosen. Do not edit the content of my resumes.

4. Command line
- python -m src.cli match
- python -m src.cli tailor --min-score 60
- python -m src.cli show-job <id>   (job, score, reasons, drafts)

Add tests for the hard filters and for the fact-check step. Stop after this
phase and summarise how to run and test it.
```

Phase 3: Application assistant

Goal	Open each approved job, fill the application form accurately and stop for your review.
Estimated time	Weeks 3 and 4 (about 8 to 12 days, the hardest phase)

What gets built

- Playwright browser session with a persistent profile, so you log in yourself.
- Form fillers for Greenhouse and Lever, plus a generic fallback.
- Field mapper that leaves uncertain fields empty and flags them.
- Pause-and-approve flow, daily limit in code, screenshots and error handling.

Done when

- On five real Greenhouse or Lever jobs the form is filled correctly and nothing is submitted without you.
- Uncertain fields are highlighted, never guessed.
- Every application has a screenshot and a status in the database.

Phase 3 prompt

Paste this after the master prompt.

```
Build PHASE 3: Application assistant (browser form filling with human approval).
Phases 1 and 2 are complete and working.

1. Browser setup (src/apply/)
- Use Playwright with Chromium in HEADED mode (visible window) so I can watch.
- Use a persistent browser profile folder so I can log in manually when a site
  needs it. Never store or type my passwords; if a login is needed, pause and
  let me log in myself.

2. Form fillers, one module per application system
- Start with Greenhouse and Lever, because their forms are standardised.
  Add a generic fallback that detects labelled inputs and maps them using
  fuzzy label matching.
- A field mapper that fills: name, email, phone, links (LinkedIn, GitHub,
  portfolio), location, resume upload (the variant chosen in Phase 2), cover
  note, and answers from the drafts and from config.yaml
  (salary, notice period, work authorization).
- Any field the mapper cannot confidently fill is left empty and highlighted on
  the page, and listed in the terminal and dashboard as "needs your input".
- Never tick legal or consent checkboxes automatically except ones I list in
  config.yaml. Never answer demographic or disability questions; leave them for me.

3. Approval flow
- python -m src.cli apply --job <id> opens the page, fills the form, scrolls to
  the Submit button, and PAUSES. It prints a summary of every value it entered.
- I review in the browser and press Submit myself. Then I confirm in the terminal
  (y/n) so the tool marks the application as applied or skipped.
- python -m src.cli apply-queue --limit 5 walks through the best drafted jobs one
  at a time with the same pause for each.
- Enforce daily_application_limit from config.yaml in code and refuse to go over.

4. Reliability
- Take a screenshot of the filled form and save it with the application record.
- On any error or unexpected page, stop, save a screenshot and a log entry, and
  move to the next job. Never retry a submit.
- Add small human-like delays between actions and between jobs. Do not attempt
  to hide that this is automation, and do not bypass CAPTCHAs: if one appears,
  pause and let me solve it.

Add tests with saved HTML fixtures of Greenhouse and Lever forms for the field
mapper. Stop after this phase and summarise how to run and test it.
```

Phase 4: Dashboard, scheduling and polish

Goal	Make the tool pleasant to use every day and ready to show on GitHub.
Estimated time	Week 5 (about 5 to 7 days)

What gets built

- Streamlit dashboard with jobs, details, today view, stats and settings.
- One-command daily run that collects, matches and drafts, but never applies by itself.
- Optional free notifications and an optional, tightly limited auto-submit for safe sources.
- Backups, error handling and a complete README with a demo.

Done when

- You can run the daily command, open the dashboard and work through the day's jobs in under 30 minutes.
- Status tracking gives a clear picture of what you applied to and what happened.
- The README is good enough that another developer can set it up from scratch.

Phase 4 prompt

Paste this after the master prompt.

```
Build PHASE 4: Dashboard, scheduling and polish. Phases 1 to 3 are complete.

1. Streamlit dashboard (app.py)
- Page "Jobs": table of matched jobs with score, verdict, company, location,
  source and status. Filters for score, status, source and remote. Sorting.
- Page "Job detail": full description, why it fits, missing skills, the cover
  note and answers (editable text boxes that save back to the database), the
  resume variant chosen, and buttons: "Open and prefill", "Skip", "Mark applied",
  "Mark interview", "Mark rejected". Add a notes field.
- Page "Today": jobs ready to apply, applications used out of the daily limit,
  and fields that need my input.
- Page "Stats": applications per week, response and interview rates, and
  most common missing skills across jobs (to guide what I should learn).
- Page "Settings": view and edit config.yaml values safely.

2. Scheduler (scheduler.py)
- A single command that runs collect, match and tailor, logs the results and
  writes a short daily summary. Document how to run it daily with Windows Task
  Scheduler or cron. It must never apply by itself.

3. Notifications (optional, free)
- A Telegram bot message or an email through Gmail SMTP with the top new matches
  each morning. Keep it behind a config flag and off by default.

4. Optional supervised auto-submit
- Add a config flag auto_submit_sources, empty by default. If I add "greenhouse"
  or "lever", the tool may press Submit on those sources only, only for jobs with
  score above a threshold I set, only when no field is flagged, and still
  respecting the daily limit. Log a screenshot before and after every submission.

5. Hardening and documentation
- Handle API quota errors gracefully and show a clear message in the dashboard.
- Back up jobs.db automatically once a day.
- Finish the README with architecture diagram, setup steps, screenshots, usage
  guide, limitations and ethical-use notes. Add a short demo script.

Stop after this phase and summarise how to run everything end to end.
```

5. Risks and how to handle them

Risk	What to do
Free tiers change or run out	Keep all AI calls behind one wrapper, cache results, and use local embeddings and Ollama as a fallback.
Websites change their HTML	Keep one module per application system and save HTML fixtures for tests. Expect small maintenance fixes.
AI invents details	Strict prompt rules, the fact-check step in Phase 2 and the [NEEDS INPUT] placeholder. Always read drafts before submitting.
Account bans	Do not automate LinkedIn, Indeed or Naukri. Use public job-board APIs and keep a human in the loop.
Privacy	Your resume goes to whichever AI provider you choose. Use Ollama locally if you want nothing to leave your laptop.
Too many weak applications	Daily limit, minimum score threshold and a review step. Fewer, better applications work better.

6. Suggested way to use this document

- Create the GitHub repository and open the project in VS Code with an AI coding assistant.
- Paste the master prompt, then paste the Phase 1 prompt. Test the result before moving on.
- Only start the next phase when the previous phase's 'Done when' list is satisfied.
- If something breaks, paste the error and the relevant file back to the assistant and ask for a fix before adding new features.
- Commit to git at the end of every phase so you can roll back.